

morning!!

HW 1? reading?

CSE/ECE 470: Computer Systems Architecture

Performance, Power

Spring 2022

Luis Ceze (Instructor)

(slides from many friends, most specially Milo Martin (UPenn) and Krste Asanovic (UC Berkeley)).

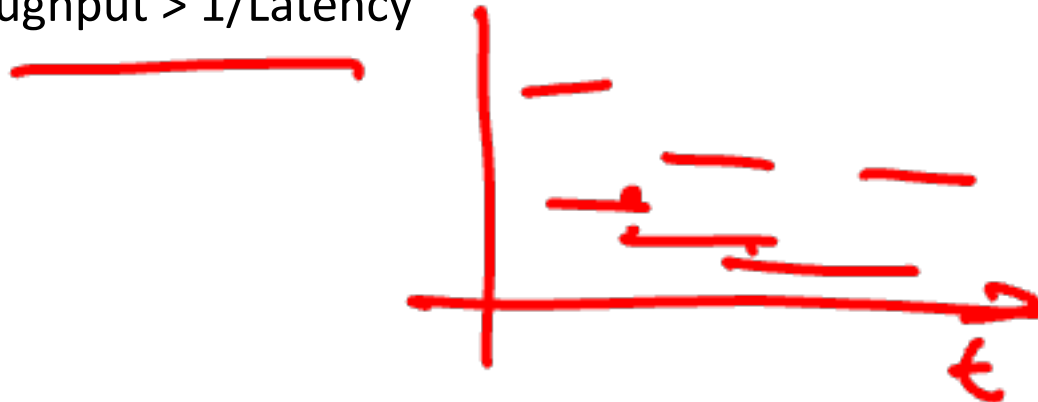
Performance

- Two common measures

- Latency (how long to do something)
 - Also called response time and execution time
- Throughput (how *often* can it do something)

- Example of car assembly line

- Takes 6 hours to make a car: latency is 6 hours
- A car leaves every 5 minutes: throughput is 12 cars per hour
- How can Throughput > 1/Latency



Comparing Performance

- **Latency:** “X is n times faster than Y”

$$\frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

- **Throughput:** “Throughput of X is n times that of Y”

$$\frac{\text{Tasks per unit time}_X}{\text{Tasks per unit time}_Y} = n$$

What is the best way to choose system X versus Y?

If Only it Were That Simple

- “X is n times faster than Y *on A*”

$$\frac{\text{Execution time of app A on machine Y}}{\text{Execution time of app A on machine X}} = n$$

- *But what about different applications*
(or even parts of the same application)
 - X is 10 times faster than Y on A, and 1.5 times on B, but Y is 2 times faster than X on C, and 3 times on D, and...

Benchmarks



- Real applications and application suites
 - E.g., SPEC CPU2000, SPEC2006, TPC-C, TPC-H, Rodinia (GPUs), MiBench (embedded/mobile), GraphBench, ...
- Kernels
 - “Representative” parts of real applications
 - Easier and quicker to set up and run
 - Often not really representative of the entire app
- Toy programs, synthetic benchmarks, etc.
 - In general, not very useful for reporting
 - Sometimes used to test/stress specific functions/features
 - E.g., memory bandwidth.

SPEC CPU (integer)

SPEC2006 benchmark description	Benchmark name by SPEC generation				
	SPEC2006	SPEC2000	SPEC95	SPEC92	SPEC89
GNU C compiler					gcc
Interpreted string processing			perl		espresso
Combinatorial optimization		mcf			li
Block-sorting compression		bzip2		compress	eqntott
Go game (AI)	go	vortex	go	sc	
Video compression	h264avc	gzip	ijpeg		
Games/path finding	astar	eon	m88ksim		
Search gene sequence	hmmer	twolf			
Quantum computer simulation	libquantum	vortex			
Discrete event simulation library	omnetpp	vpr			
Chess game (AI)	sjeng	crafty			
XML parsing	xalancbmk	parser			

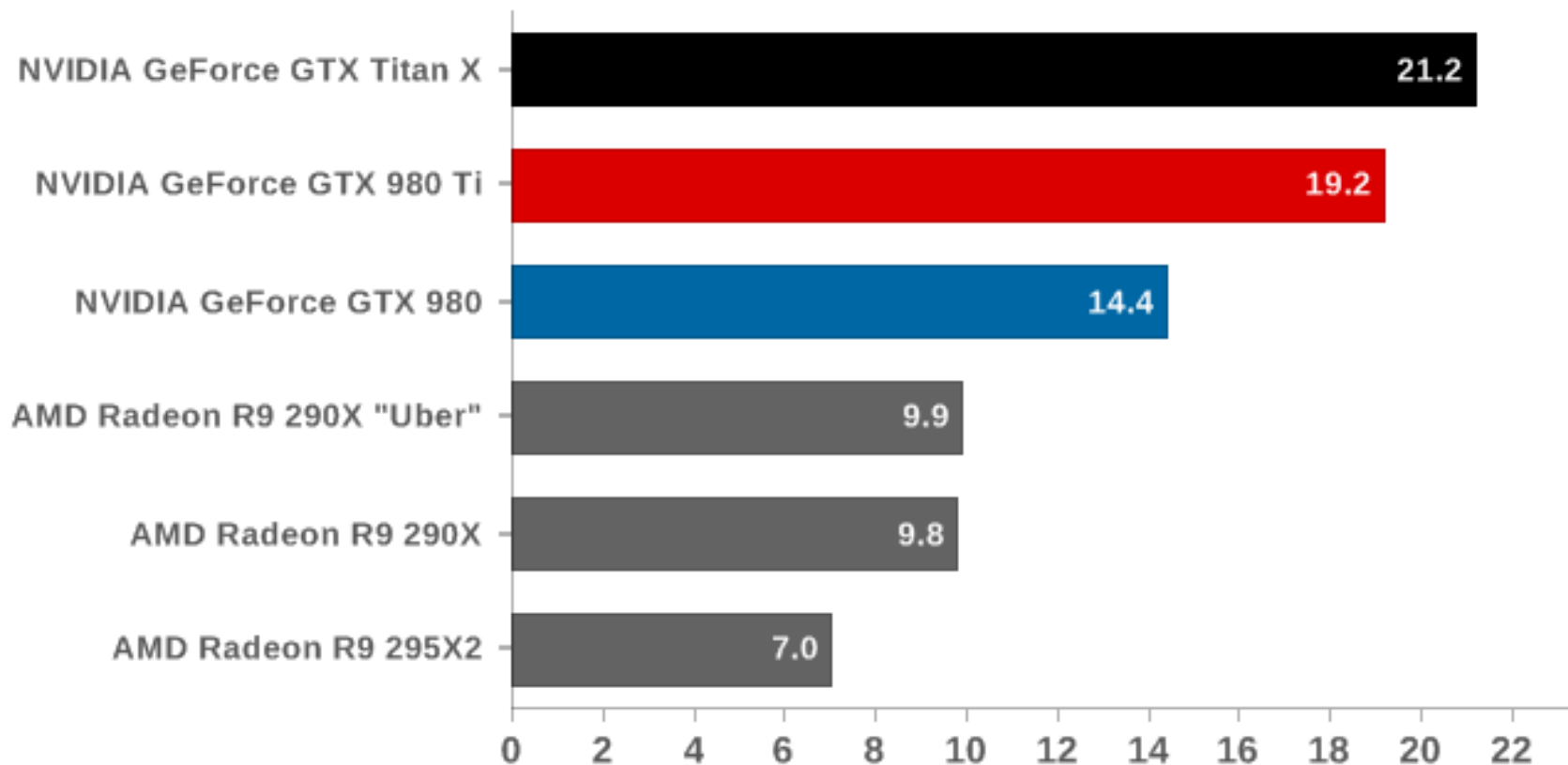
"Representative" applications keeps growing with time!

SPEC Cloud IaaS 2016 Benchmark: Scalability, Elasticity, Mean instance provisioning time.



Grand Theft Auto V - 99th Percentile Framerate - 3840x2160 - Very High Quality

Minimum Frames Per Second - Higher Is Better



CPU Performance Equation (1)

$$\text{CPU time} = \text{CPU Clock Cycles} \times \text{Clock cycle time}$$

$$\text{CPU time} = \text{Instruction Count} \times \text{Cycles Per Instruction} \times \text{Clock cycle time}$$

Diagram showing the breakdown of the CPU time equation:

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

Arrows indicate the mapping from the high-level equation to the unit-based equation:

- Instruction Count $\rightarrow$ Instructions / Program
- Cycles Per Instruction $\rightarrow$ Clock Cycles / Instruction
- Clock cycle time $\rightarrow$ Seconds / Clock Cycle

ISA
Algo
Compiler

Microarchitecture
ISA

Arch
Process Tech

What affects each of these?

CPU Performance Equation (1)

$$\text{CPU time} = \text{CPU Clock Cycles} \times \text{Clock cycle time}$$

$$\text{CPU time} = \overbrace{\text{Instruction Count} \times \text{Cycles Per Instruction} \times \text{Clock cycle time}}$$

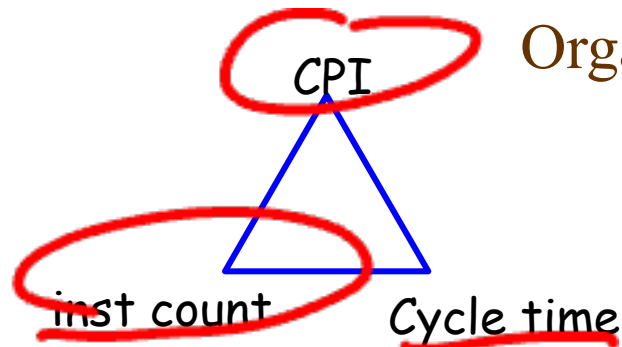
$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

ISA,
Compiler
Technology

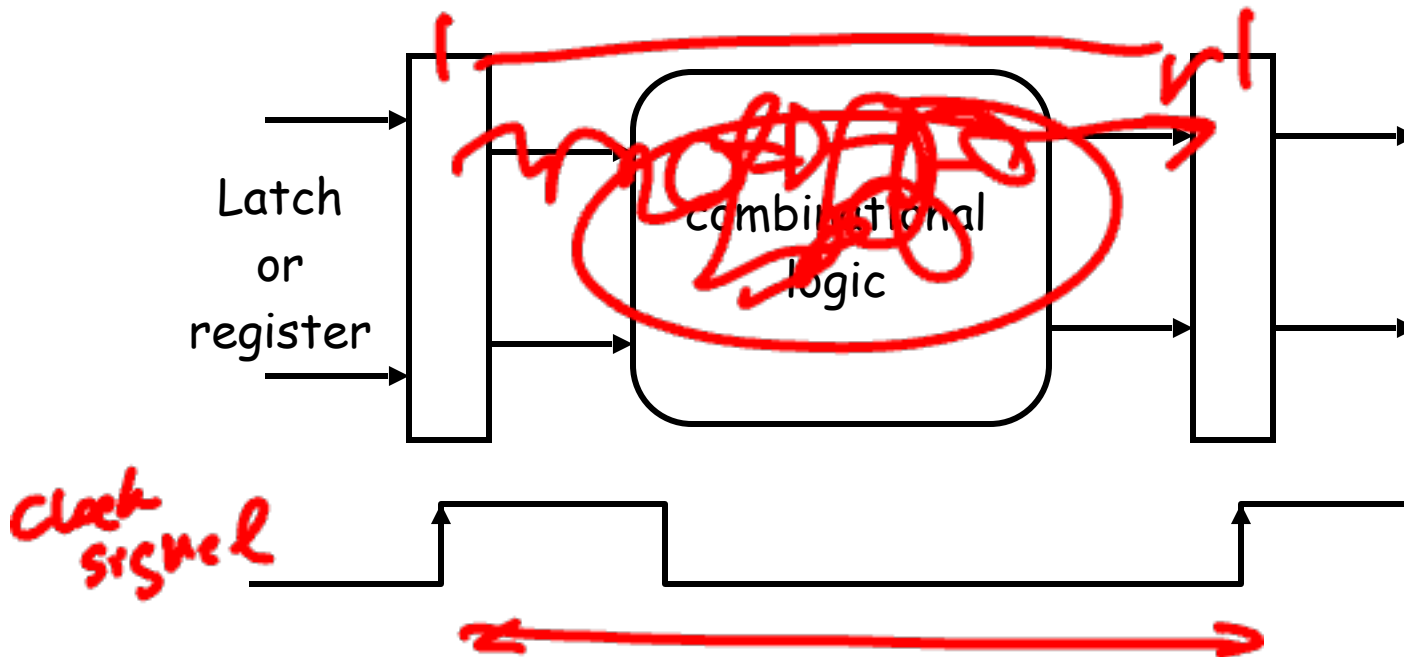
Organization,
ISA

Hardware
Technology,
Organization

“Iron Law of Performance”



What's a Clock Cycle?



- Determined by numerous time-of-flight issues + gate delays
 - clock propagation, wire lengths, repeaters
 - Can get *super complicated* -- design tools!
- ~ 8-12 classic gate delays common

Example

- Program takes 33 billion instructions to run
- CPU processes insts at 2 cycles per inst
- Clock speed of 3GHz

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{33 \times 10^9 \text{ Instructions}}{\text{Program}} \times \frac{2 \text{ Clock Cycles}}{\text{Instruction}} \times \frac{1 \text{ Seconds}}{3 \times 10^9 \text{ Clock Cycle}} = \frac{66}{3}$$

Sometimes clock cycle time given instead (ex. cycle = 333 ps)

IPC sometimes used instead of CPI

= 22 seconds

CPU Performance Equation (1)

$$\text{CPU time} = \text{CPU Clock Cycles} \times \text{Clock cycle time}$$

$$\text{CPU time} = \text{Instruction Count} \times \text{Cycles Per Instruction} \times \text{Clock cycle time}$$

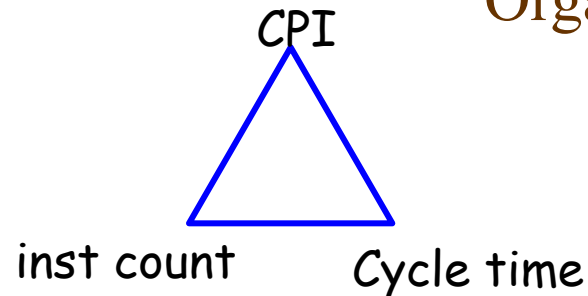
$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

ISA,
Compiler
Technology

Organization,
ISA

Hardware
Technology,
Organization

“Iron Law of Performance”



CPU Performance Equation (2)

CPU time = CPU Clock Cycles $\times$ Clock cycle time

$$\text{CPU time} = \left(\sum_{i=1}^n \text{IC}_i \times \text{CPI}_i \right) \times \text{Clock cycle time}$$

For each kind
of instruction

How many cycles it
takes to execute an
instruction of this kind

How many instructions
of this kind are there in
the program

Calculating CPI

- Computed from instruction mix and CPI of each instruction type
- Very important quantitative metric

$$CPI = \sum_{i=1}^N \frac{IC_i}{IC} CPI_i$$

ALU	50%	1
Branches	15%	2
Loads	20%	2
Stores	15%	1

$$CPI = .5 \times 1 + .15 \times 2 + .2 \times 2 + .15 \times 1 = 1.35$$

Which processor would you buy?

Intel

- Processor A: CPI = 2, clock = 5 GHz

AMD

- Processor B: CPI = 1, clock = 3 GHz

Which processor would you buy?

- Processor A: CPI = 2, clock = 5 GHz
- Processor B: CPI = 1, clock = 3 GHz
- Probably A, but B is faster... assuming same ISA and same compiler.
- Classic example
 - ~~800 MHz PentiumIII faster than 1 GHz Pentium4!~~
 - Same ISA and compiler!
- **Meta-point: danger of partial performance metrics!**

Summarizing Performance

- Arithmetic mean
 - Average execution time
 - Gives more weight to longer-running programs
- Weighted arithmetic mean
 - More important programs can be emphasized
 - But what do we use as weights?
 - Different weight will make different machines look better

Speedup

	Machine A	Machine B
Program 1	5 sec	4 sec
Program 2	3 sec	6 sec

→ What is the speedup of A compared to B on Program 1? $\frac{T_B}{T_A} = \frac{4}{5} = 0.8x$

What is the speedup of A compared to B on Program 2? $\frac{T_B}{T_A} = \frac{6}{3} = 2x$

What is the average speedup? $\frac{2 + 0.8}{2} = 1.4x$

What is the speedup of A compared to B on Sum(Program1, Program2) ? $\frac{T_B}{T_A} = \frac{10}{8} = 1.25x$

Speedup

	Machine A	Machine B
Program 1	5 sec	4 sec
Program 2	3 sec	6 sec

What is the speedup of A compared to B on Program 1? **0.8x**

What is the speedup of A compared to B on Program 2? **2x**

What is the average speedup? **1.4x**

What is the speedup of A compared to B on Sum(Program1, Program2) ? **1.25x**

Mean (Average) Performance Numbers

- **Arithmetic:** $(1/N) * \sum_{P=1..N} \text{Latency}(P)$
 - For units that are proportional to time (e.g., latency)
- You can add latencies (time), but not throughputs (rate/time)
 - $\text{Latency}(P1+P2,A) = \text{Latency}(P1,A) + \text{Latency}(P2,A)$
 - $\text{Throughput}(P1+P2,A) \neq \text{Throughput}(P1,A) + \text{Throughput}(P2,A)$
 - 1 mile @ 30 miles/hour + 1 mile @ 90 miles/hour
 - Average is **not** 60 miles/hour
- **Harmonic:** $N / \sum_{P=1..N} 1/\text{Throughput}(P)$
 - For units that are inversely proportional to time (e.g., throughput)
- **Geometric:** $N\sqrt[N]{\prod_{P=1..N} \text{Speedup}(P)}$
 - For unitless quantities (e.g., speedup ratios)
- Suggested reading: Smith on Summarizing performance with a single number... READ IT 😊

Optimizing for performance

- What should you go after first?
 - E.g., what part of your program should you parallelize?



- What is the limit of your performance optimization?



2x

Amdahl's Law (1)

$$\text{Speedup} = \frac{\text{Execution Time without Enhancement}}{\text{Execution Time with Enhancement}} = \frac{\text{Execution Time}_{\text{old}}}{\text{Execution Time}_{\text{new}}}$$

What if enhancement does not enhance everything?

$$\text{Speedup} = \frac{\text{Execution Time without using Enhancement at all}}{\text{Execution Time using Enhancement when Possible}}$$

$$\text{Execution Time}_{\text{new}} = \text{Execution Time}_{\text{old}} \times \left((1 - \text{Fraction}_{\text{Enhanced}}) + \frac{\text{Fraction}_{\text{Enhanced}}}{\text{Speedup}_{\text{Enhanced}}} \right)$$

$$\text{Overall Speedup} = \frac{1}{\left((1 - \text{Fraction}_{\text{Enhanced}}) + \frac{\text{Fraction}_{\text{Enhanced}}}{\text{Speedup}_{\text{Enhanced}}} \right)}$$

Caution: fraction
of What?

Amdahl's Law (2)

- Make the Common Case Fast

$$\text{Overall Speedup} = \frac{1}{\left((1 - \text{Fraction}_{\text{Enhanced}}) + \frac{\text{Fraction}_{\text{Enhanced}}}{\text{Speedup}_{\text{Enhanced}}} \right)}$$

Speedup_{Enhanced} = 20 Fraction_{Enhanced} = 0.1 VS Speedup_{Enhanced} = 1.2 Fraction_{Enhanced} = 0.9

$$\text{Speedup} = \frac{1}{\left((1 - 0.1) + \frac{0.1}{20} \right)} = 1.105$$

$$\text{Speedup} = \frac{1}{\left((1 - 0.9) + \frac{0.9}{1.2} \right)} = 1.176$$

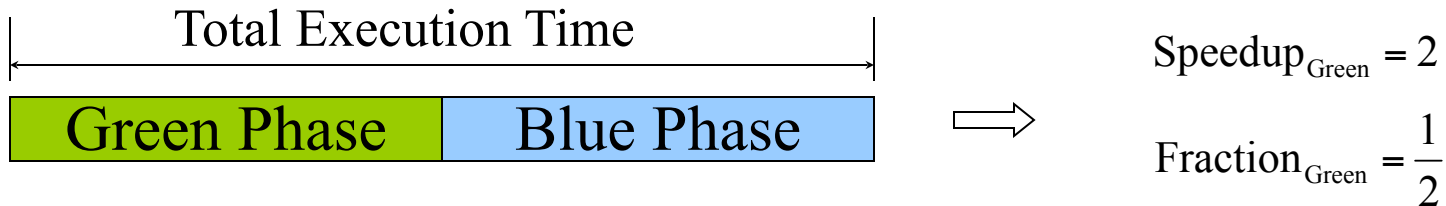
Important: Principle of locality

Approx. 90% of the time spent in 10% of the code

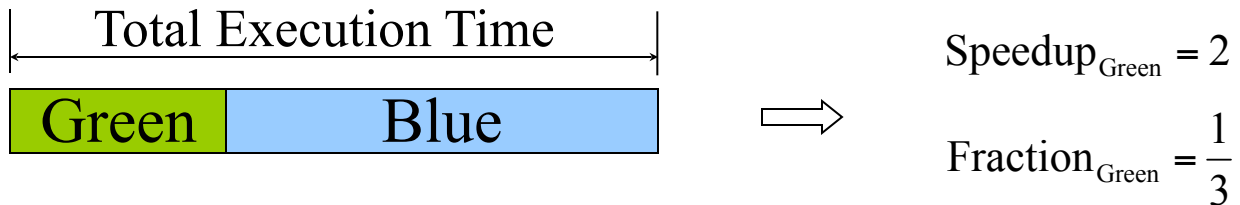
Amdahl's Law (3)

- Diminishing Returns

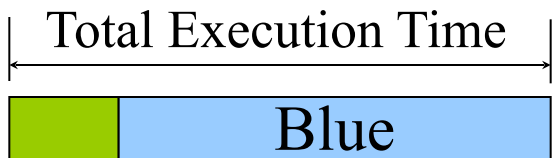
Generation 1



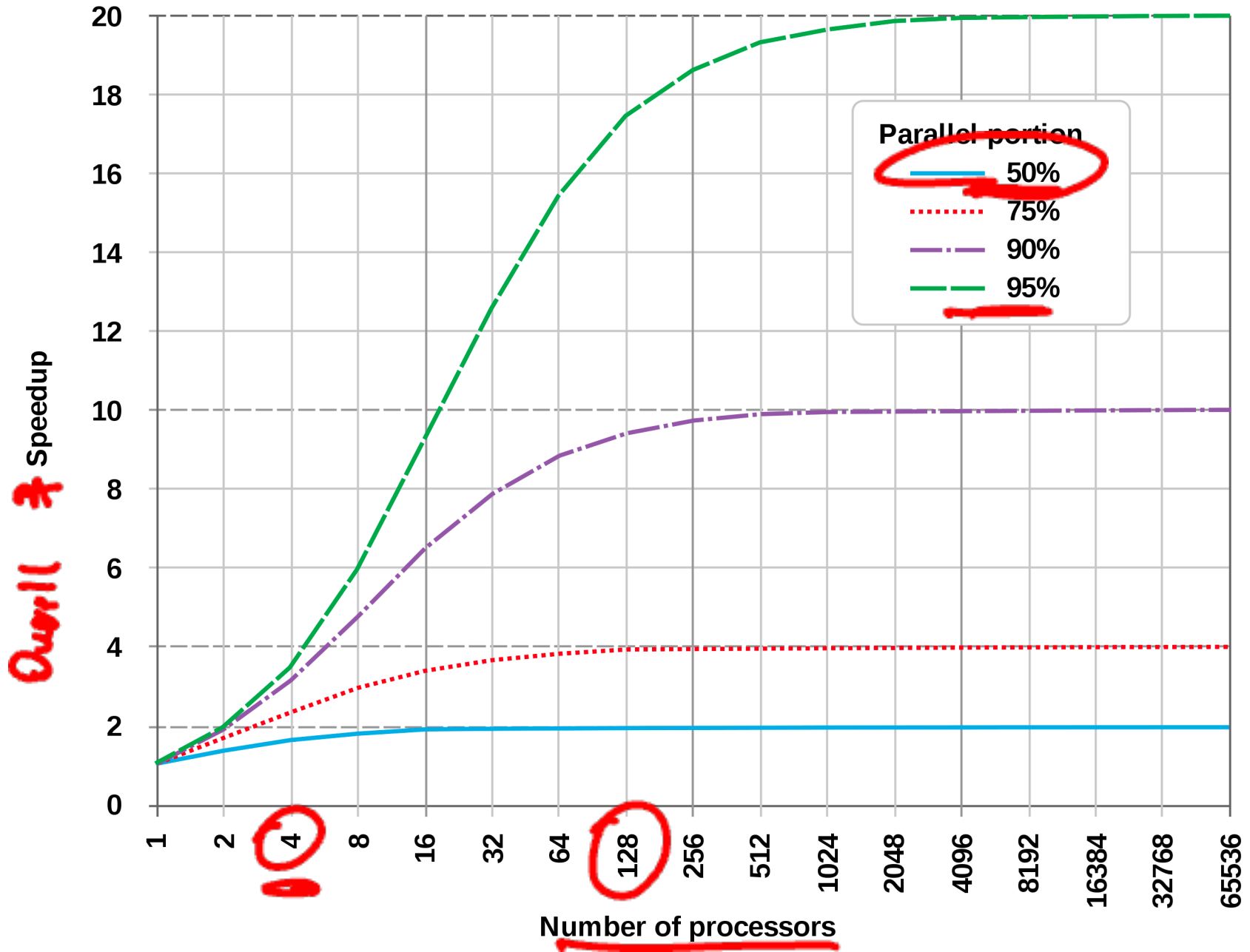
Generation 2 Speedup_{Overall} = 1.33 over Generation 1



Generation 3 Speedup_{Overall} = 1.2 over Generation 2



Amdahl's Law



Rules of Thumb

- Make the common case fast
 - driving force behind the RISC philosophy
 - easier for compilers to optimize, simpler decoding
 - Design for actual performance, not peak performance
 - Amdahl's law
- Locality of reference (90/10 rule)
 - programs spend 90% of their time in 10% of the code
 - main principle behind caches (spatial/temporal locality)
- Smaller is faster
 - Why?
 - main principle behind memory hierarchies
 - give illusion of fast, large memory

NPC

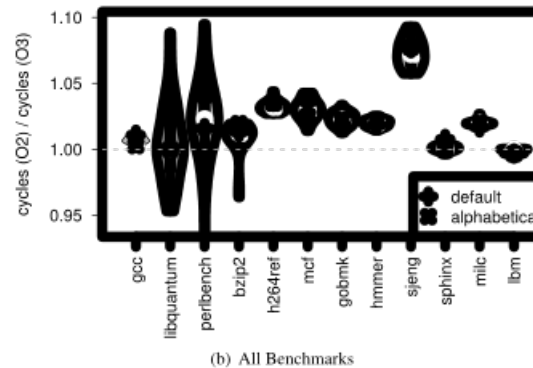
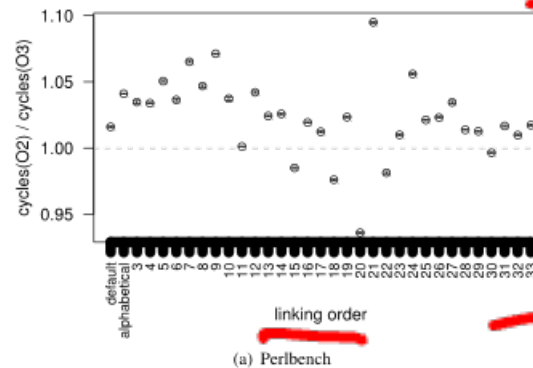
Producing Wrong Data Without Doing Anything Obviously Wrong!

Todd Mytkowicz Amer Diwan
Department of Computer Science
University of Colorado
Boulder, CO, USA
{mytkowit,diwan}@colorado.edu

Matthias Hauswirth
Faculty of Informatics
University of Lugano
Lugano, CH
Matthias.Hauswirth@unisi.ch

Peter F. Sweeney
IBM Research
Hawthorne, NY, USA
pfs@us.ibm.com

Auto tuning



Power/Energy

- Why are they important?

Battery life

Cooling

Cost $\leadsto$

Reliability

Fan $\sim 10-20W$ USB $\sim 50W$

Screen $\sim 10W$ Disk/BSD

GPU $\sim 200W$

$\sim 100W$

CPU $\sim 100W$

DRAM $\sim 100-200W$

WAT stable $\sim \$0.20 kWh$

$\sim 1kW$

Energy != Power

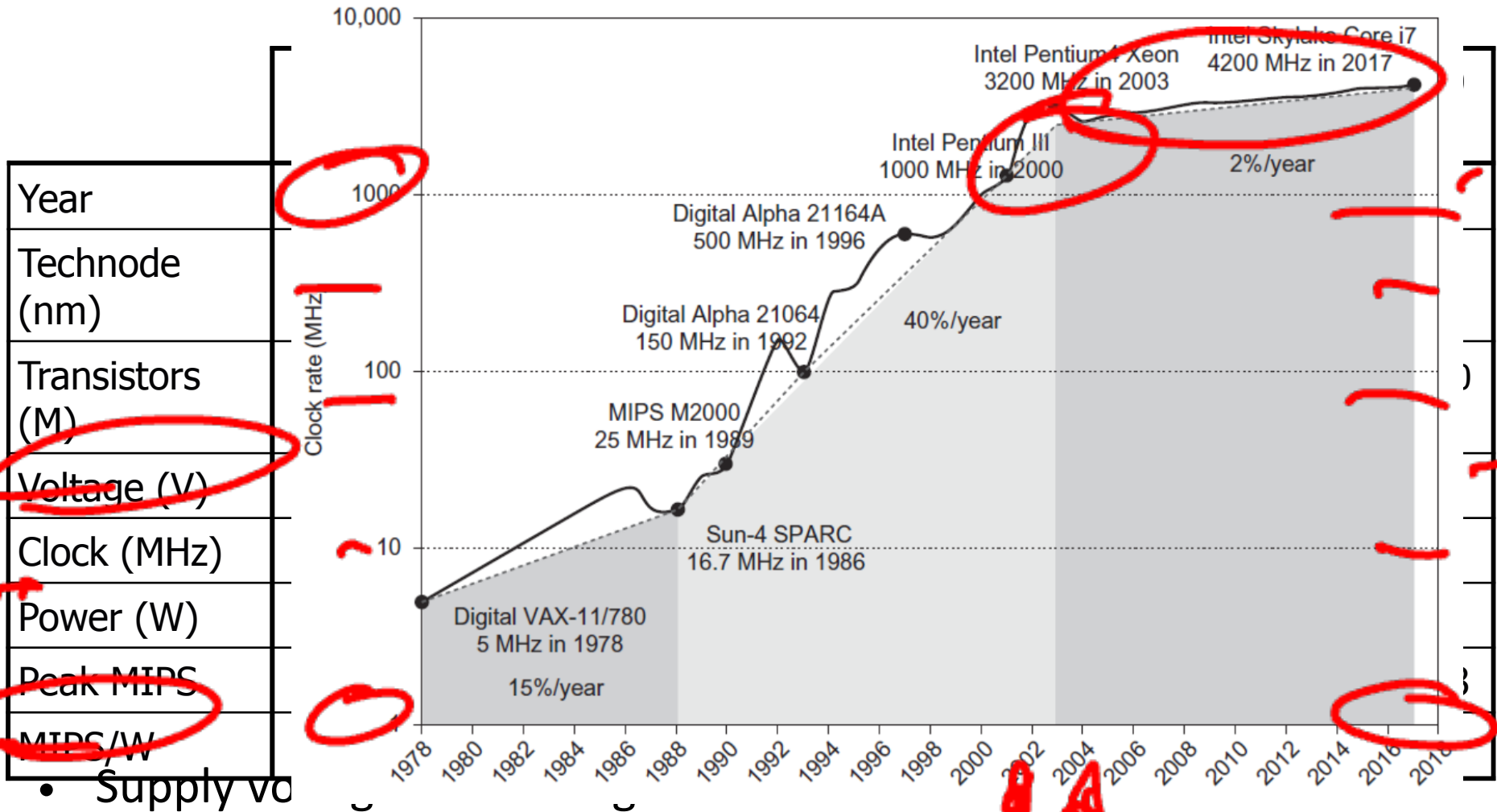
$$\underline{E = P \times t}$$

- **Energy**: measured in Joules or Watt-seconds
 - Total amount of energy stored/used
 - Battery life, electric bill, environmental impact
 - Instructions per Joule (car analogy: miles per gallon)
- **Power**: energy per unit time (measured in Watts)
 - Related to “performance” (which is also a “per unit time” metric)
 - Power impacts power supply and cooling requirements (cost)
 - Power density (Watt/mm²): important related metric
 - Peak power vs average power
 - E.g., camera, power “spikes” when you actually take a picture
 - Joules per second (car analogy: gallons per hour)
- Two sources:
 - **Dynamic power**: active switching of transistors
 - **Static power**: leakage of transistors even while inactive

Power/Energy: Increasingly Important

- **Battery life** for mobile devices
 - Laptops, phones, cameras
 - Size too!
- **Tolerable temperature** for devices without active cooling
 - Power means temperature, active cooling means **cost**
 - No room for a fan in a cell phone, no market for a hot cell phone
- **Electric bill** for compute/data centers
 - Pay for power twice: once in, once out (to cool)
- **Design constraints in data-centers:** cost proportional to peak power
- **Environmental concerns**
 - “Computers” account for growing fraction of energy consumption

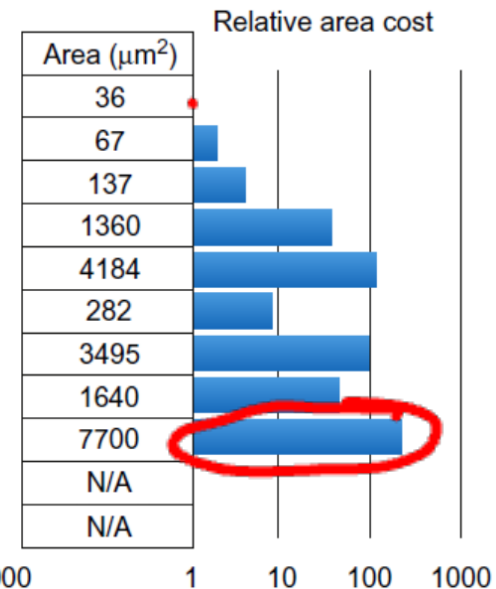
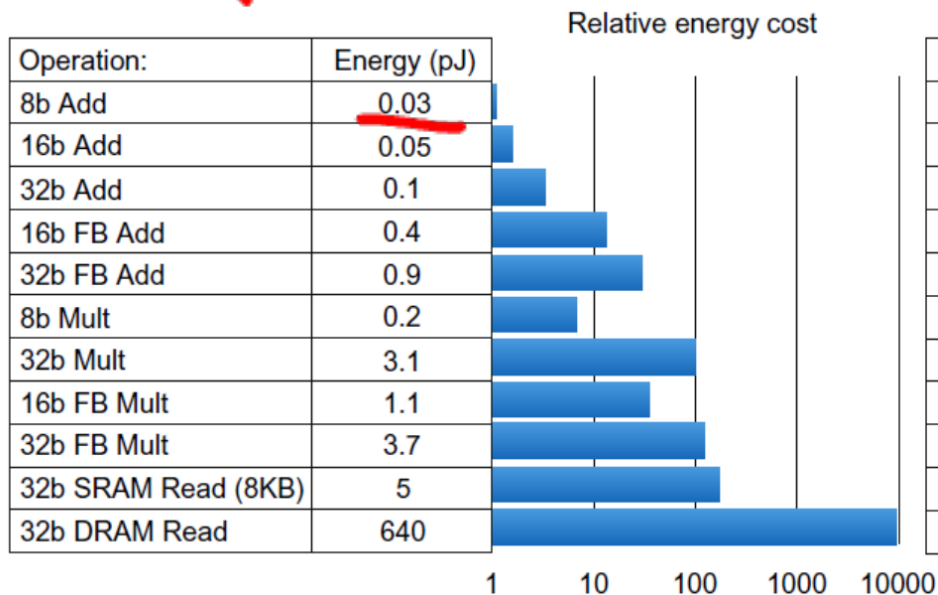
Trends in Power



- Emphasis on power starting around 2000
 - Resulting in slower frequency increases

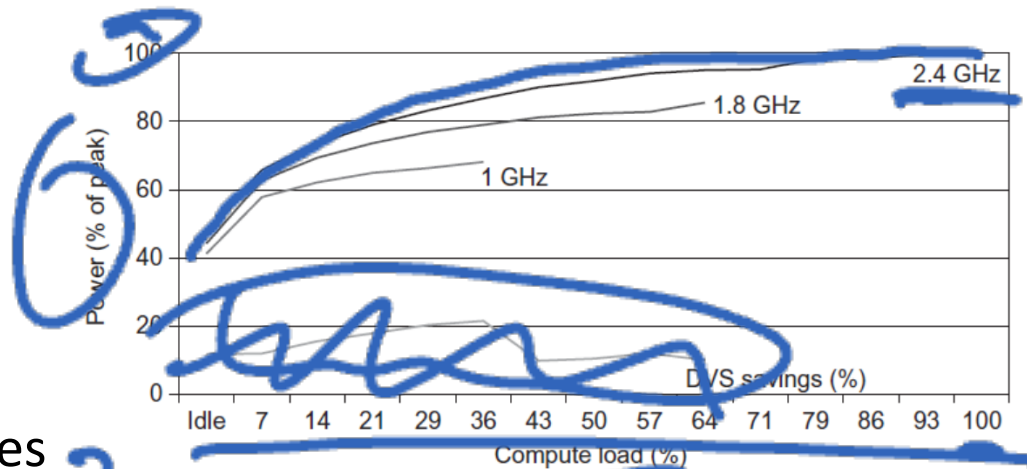
Static Power

- Static power consumption
 - 25-50% of total power
 - $\text{Current}_{\text{static}} \times \text{Voltage}$
 - Scales with number of transistors
 - To reduce: power gating



Saving Power

- DVFS *dynamic voltage frequency scaling*
- Power savings mode
- Low power state for:
 - DRAM, disks
- Overclocking, turning off cores
- Parallelism? Why?
- Other software tricks?
 - Embedded systems
 - Data-centers
- Other ideas?
- First paper reading assignment: Power as first class concern



Two Approaches

$$P \sim V f^2 \sim f^3$$

$$2 \times \left(\frac{1}{2}\right)^3$$

$$\boxed{f}$$

$$\boxed{\frac{1}{2}}$$

$$\boxed{\frac{1}{4}}$$

$$\frac{2f^3}{8}$$

$$\frac{1}{4} f^3$$